

Objetivo

Entrenar y comparar modelos de clasificación de imágenes de lesiones de piel (9 clases) usando PyTorch, explorando arquitecturas (MLP y CNN), regularización, inicialización de pesos y búsqueda automática de hiperparámetros. Todo el proceso se logueó con TensorBoard y MLflow.

Dataset

Se usó el dataset Split_smol del repositorio del curso. El EDA mostró desbalance de clases y resoluciones heterogéneas, lo que motivó el uso de A.Resize() y A.Normalize() en todos los experimentos.

Notebook 1 — MLP Simple

Se entrenó un MLP de 3 capas densas (512 → 128 → 9) como baseline, explorando dos aspectos.

Regularización

Configuración	Dropout	BatchNorm	Weight Decay	Val Accuracy
baseline	0.0	No	0.0	56.91%
dropout_only	0.5	No	0.0	48.62%
batchnorm_dropout	0.5	Sí	0.0	58.01%
batchnorm_dropout_wd	0.5	Sí	1e-4	59.67%

La combinación completa (BatchNorm + Dropout + Weight Decay) dio el mejor resultado. Un hallazgo interesante: Dropout sin BatchNorm (48.62%) fue peor que el baseline sin ninguna regularización (56.91%), lo que sugiere que el ruido de Dropout sin la estabilización de BatchNorm dificultó la convergencia en pocas épocas.

Inicialización

Init	Val Accuracy
default	51.4%
uniform	53.0%
he	53.6%
xavier	55.8%

Xavier dio el mejor resultado en este caso particular, aunque He arrancó con mejor accuracy en la primera época (42.0% vs 40.9%), reflejando su diseño específico para ReLU. La diferencia entre todas las inicializaciones fue moderada (~5 puntos), probablemente porque BatchNorm amortigua el impacto de una inicialización subóptima.

Notebook 2 — MLP con Búsqueda de HP

Se realizó una búsqueda aleatoria sobre 1728 combinaciones posibles de HPs (input_size, batch_size, lr, optimizer, augmentations, dropout), entrenando 26 modelos (~1.5% del espacio).

Mejor modelo encontrado:

- input_size=64, batch_size=64, lr=0.0001, optimizer=Adam, dropout=0.0
- Val Accuracy: 67.96%

El optimizer Adam dominó el top 10 (8 de 10 mejores resultados), y la mayoría de los mejores modelos tuvieron dropout=0.0, probablemente porque el entrenamiento corto (con early stopping) no le dio tiempo a Dropout de mostrar su beneficio en validación.

Notebook 3 — CNN con Búsqueda de HP

Se reemplazó el MLP por una CNN de 3 bloques convolucionales (32→64→128 filtros, cada uno con BatchNorm2d + ReLU + MaxPool) con AdaptiveAvgPool2d para soportar cualquier input_size. Se repitió la búsqueda aleatoria con un espacio reducido (384 combinaciones), entrenando 20 modelos.

Mejor modelo encontrado:

- input_size=64, batch_size=64, lr=0.001, optimizer=Adam, dropout=0.0
- Val Accuracy: 76.24%

Comparación General

Modelo	Val Accuracy	Mejor lr	Mejor batch_size	Mejor optimizer
MLP (búsqueda HP)	67.96%	0.0001	64	Adam
CNN (búsqueda HP)	76.24%	0.001	64	Adam

La CNN superó al MLP por 8.28%, usando exactamente el mismo espacio de búsqueda de hiperparámetros. Esto confirma la ventaja teórica de las convoluciones para datos con estructura espacial: la CNN explota la localidad de los píxeles mediante filtros con pesos compartidos, mientras que el MLP trata cada píxel como una feature independiente, requiriendo muchos más parámetros para aprender patrones equivalentes.

Ambos modelos coincidieron en preferir batch_size=64 y el optimizer Adam, sugiriendo que estos HPs son relativamente robustos para este dataset independientemente de la arquitectura.

Conclusiones

- El pipeline de clasificación con PyTorch + Albumentations + MLflow + TensorBoard permitió experimentar de forma ordenada y comparar resultados de manera sistemática.
- La combinación BatchNorm + Dropout + Weight Decay redujo el overfitting de forma consistente, mostrando la brecha train-val más saludable de todas las configuraciones probadas.
- El efecto de la inicialización de pesos fue moderado en presencia de BatchNorm, que compensa parcialmente una inicialización subóptima.
- La CNN superó claramente al MLP (76.24% vs 67.96%) con la misma búsqueda de hiperparámetros, confirmando que la estructura convolucional es más adecuada para clasificación de imágenes.
- El learning rate y el optimizer fueron los HPs más influyentes en ambas búsquedas; Adam con lr bajo ($1e-4$ a $1e-3$) y batch_size=64 fue la combinación más robusta en ambos modelos.